

# Code correcteur

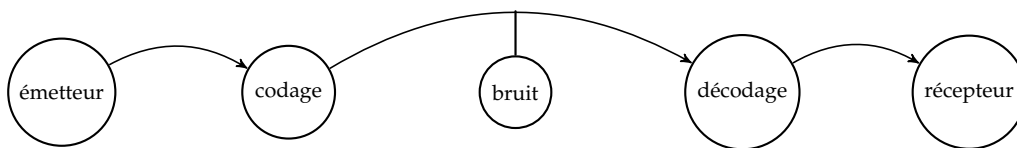


Lycée Faidherbe  
2014-2015

## 1 Introduction

Les codes correcteurs d'erreurs sont utilisés dans les transmissions de données pour éliminer les effets du bruit. Les données sont ici des listes de 0 et de 1. On propose de construire plusieurs schémas simples de codes correcteurs allant du très simple bit de parité au code de Hamming.

Un canal de communication peut être schématisé comme sur la Figure 1.



La transmission est perturbée par un bruit et un système de codage est utilisé pour détecter, ou même corriger les erreurs provoquées par le bruit. C'est ce qu'on appelle un code correcteur d'erreurs. Ce mécanisme est utilisé sur tous les moyens de transmission de données informatiques (cables coaxiaux, fibres optiques, ondes hertziennes, etc.).

## 2 Bit de parité

Le mécanisme le plus simple est l'utilisation d'un bit de parité.

Il consiste à ajouter à la fin de chaque bloc de  $n - 1$  bits un bit supplémentaire choisi de telle sorte que le nombre total de bits égaux à 1 dans le bloc de  $n$  bits soit pair. Si, à la réception, le nombre de bits dans un bloc est impair, on détecte une erreur (et on peut, par exemple envoyer un message demandant la retransmission du bloc). Ce principe est aussi exploité dans la norme ASCII afin de mieux différencier les codes affectés à chaque caractère.

1. Écrire une fonction `parite(liste)` qui retourne 0 si le nombre de 1 est pair dans cette liste et 1 sinon. On admet que `liste` est une liste qui ne peut contenir que des 0 et des 1.
2. Écrire une fonction `codage(liste,n,p)` qui prend en entrée une liste qui ne peut contenir que des 0 et des 1 et qui retourne un tableau numpy de taille  $n \times p$  tel que sa dernière colonne contient le bit de parité de chacune de ses lignes tronquées à la dernière colonne. Si on concatène les lignes (en excluant les termes de la colonne  $p - 1$ ), on retrouve `liste`. On admet que `liste` est de longueur  $n \times (p - 1)$ .

Par exemple `codage([0,1,1,0,0,1],2,4)` retourne  $\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$ .

Pour se rafraîchir la mémoire on pourra taper dans l'éditeur :

```

import numpy as np
t=np.array([[8,5],[7,0]])
print(t)
y=np.zeros((2,4),int)
print(y)
print(y.shape)
l=[6,4,3,1,5,2]
y[0,1:3]=l[2:4]
print(y)
b=np.array([[2],[4],[6],[8]])
print(np.dot(y,b))
enfin=np.transpose(b)
print(enfin)

```

3. Écrire une fonction `verification(tableau)` qui prend en entrée un tableau numpy qui ne peut contenir que des 0 et des 1. Si la dernière colonne du tableau contient le bit de parité de chacune de ses lignes tronquées à la dernière colonne, alors la fonction retourne  $-1$ . Sinon elle retourne le numéro  $i$  de la première ligne où la dernière colonne du tableau ne contient pas le bit de parité de la ligne  $i$  tronquée.  
 Tester avec `t1=np.array([[0,1,0,1],[1,1,1,0]])` et `t2=np.array([[0,1,0,1],[1,1,0,0]])`.

### 3 Code de Hamming

Le bit de parité permet de détecter une erreur mais n'indique pas la position précise de l'erreur. On notera aussi que deux erreurs dans le même bloc passent inaperçues car elles ne modifient pas la parité.

#### 3.1 Encodage

Un procédé plus compliqué permet de corriger une erreur dans un bloc (mais pas deux et le procédé est donc valable pour des taux d'erreurs faibles). Le code utilisé, appelé *code de Hamming*, est obtenu de la façon suivante. On fixe un entier  $k$  et on code chaque bloc de  $m = 2^k - k - 1$  bits par un bloc de  $n = 2^k - 1$  bits obtenu en ajoutant à certaines positions dans ce bloc de  $m$  bits (dits bits d'information) un ensemble de  $k$  bits de correction.

Pour calculer les bits de correction, on utilise une matrice de parité  $H$  qui est représentée ci-dessous pour  $k = 3$ .

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \in \mathcal{M}_{(k,n)}(\mathbb{Z}/2\mathbb{Z}).$$

La colonne numéro  $j \in \llbracket 1, 7 \rrbracket$  de  $H$  est la représentation en binaire de  $j$ . Par exemple  $1 = 1 \times 2^0 + 0 \times 2^1 + 0 \times 2^2$  et  $6 = 0 \times 2^0 + 1 \times 2^1 + 1 \times 2^2$ .

Les  $n - m = k$  bits de correction sont choisis de telle façon que si le bloc transmis  $a$  contenant des 0 et des 1 est écrit comme un vecteur colonne de longueur  $n = 2^3 - 1 = 7$ , on ait  $Ha = (0)$ . Les opérations sont faites en binaire avec la règle  $1 + 1 = 0$  (c'est le corps noté  $\mathbb{Z}/2\mathbb{Z}$  des entiers modulo 2). Les bits de correction sont ceux situés aux indices puissance de 2 (colonnes 1, 2 et 4).

Par exemple, on veut transmettre  $b = [0, 1, 1, 0]$  de longueur  $n - k = 7 - 3 = 4$ . On ajoute trois bits de correction en position 1, 2 et 4 pour obtenir  $a = [a_0, a_1, 0, a_2, 1, 1, 0]$ . Les  $a_i$

vérifient

$$H \begin{pmatrix} a_0 \\ a_1 \\ 0 \\ a_2 \\ 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

On obtient trois équations à trois inconnues. La position stratégique des  $a_i$  assure qu'il

n'y en a qu'un seul par équation. Pour les calculer il suffit de faire le produit,  $H \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$ . Si

sur une ligne  $i \in \llbracket 1, 3 \rrbracket$  on a 0 alors  $a_{i-1} = 0$  sinon  $a_{i-1} = 1$ .

4. Écrire une fonction `codageHamming(b)` qui retourne le tableau numpy  $a = [a_0, a_1, b_0, a_2, b_1, b_2, b_3]$  avec le paramètre  $b = [b_0, b_1, b_2, b_3]$ .

Vérifier que `codageHamming([0,1,1,0])` donne `array([1, 1, 0, 0, 1, 1, 0])`.

5. **À faire en deuxième lecture :**

(a) Écrire une fonction `matriceTrans(k)` qui retourne le tableau numpy  $H$  formées de  $2^k - 1$  colonnes et  $k$  lignes où chaque colonne de numéro  $j \in \llbracket 1, 2^k - 1 \rrbracket$  de  $H$  est la représentation en binaire de  $j$ . Le bit de poids faible (celui de l'unité) est sur la première ligne. Vérifier avec  $k = 3$ .

(b) Écrire une fonction `codageHam2(k, b)` qui retourne le tableau numpy (ou la liste)  $a = [a_0, a_1, b_0, a_2, b_1, b_2, b_3, \dots]$  de longueur  $n = 2^k - 1$  qui est le codage de Hamming de la liste  $b = [b_0, \dots, b_{n-k-1}]$  avec l'entier  $k$ .

Vérifier que `codageHam2(3, [0,1,1,0])` donne `[1, 1, 0, 0, 1, 1, 0]`.

### 3.2 Décodage

Supposons qu'il y ait une erreur sur le deuxième bit, on reçoit le bloc  $c = a + e$  avec  $e = [0, 1, 0, 0, 0, 0, 0]$ . Elle est corrigée au décodage de la façon suivante. On calcule le vecteur  $s = H^t c$ . On obtient ainsi le vecteur  $H^t e$  qui est la deuxième colonne de  $H$  et on change donc le deuxième bit de  $c$  de 1 en 0.

Le code de Hamming permet de corriger une erreur car toutes les colonnes de la matrice  $H$  sont différentes et que deux blocs codés diffèrent au moins sur trois bits. Il ne permet pas de corriger ni même de détecter deux erreurs. En effet, si deux erreurs se produisent aux indices  $i$  et  $j$ , le résultat est le même que si une erreur s'était produite (dans un autre bloc) à l'indice  $k$  tel que la colonne  $k$  de la matrice  $H$  soit la somme binaire de colonnes  $i$  et  $j$ .

6. On suppose que le tableau numpy  $a$  est codé selon Hamming et qu'il contient au plus une erreur. On note  $h$  le tableau numpy de transition. Écrire une fonction `decodageHam(h, a)` qui retourne  $a$  s'il n'y a pas d'erreur et une version corrigée sinon. On pourra utiliser la conversion en liste pour comparer un vecteur colonne et une colonne d'une matrice avec le mot clé `list` (colonne numpy).

## 4 Code de Hamming cyclique

On garde  $n = 2^k - 1$ .

Une variante de la méthode ci-dessus consiste à considérer un vecteur binaire  $b = [b_i]_{0 \leq i \leq k-1}$  comme un polynôme  $b(X) = \sum_{i=0}^{k-1} b_i X^i$  à coefficients dans  $\mathbb{Z}/2\mathbb{Z}$ . On utilise un polynôme  $p(X)$  de degré  $k$  qui est *primitif*, c'est-à-dire tel que les restes des monômes  $(X^i)_{0 \leq i \leq n-1}$  de la division euclidienne par  $p$  (on dira modulo  $p$ ) parcourent l'ensemble de tous les polynômes non nuls de degré au plus  $k$  et à coefficients dans  $\mathbb{Z}/2\mathbb{Z}$  une fois et une seule.

Par exemple, si on prend  $p(X) = 1 + X + X^3$ , on obtient, en écrivant les vecteurs  $(X^i)_{0 \leq i \leq 6}$  modulo  $p$  en colonnes dans la base  $(1, X, X^2)$ , la matrice

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Le polynôme  $p$  est donc primitif. C'est la matrice  $H$  ci-dessus que l'on prend comme matrice de parité. On notera que, par linéarité du calcul du reste, pour tout bloc  $v$  de longueur  $n = 2^k - 1$ , le produit  $Hv$  est la matrice du reste modulo  $p$  de  $v(X)$  dans la base  $(X^i)_{0 \leq i \leq k-1}$ .

Pour transmettre  $a' = [a_i]_{0 \leq i \leq n-k-1}$ . On pose  $a = [0, \dots, 0, a_0, \dots, a_{n-k-1}]$  avec  $k = n - m$  bits de valeur 0 en tête destinés à la correction. Le calcul des bits de correction consiste donc cette fois tout simplement à calculer le polynôme  $b(X) = a(X) + r(X)$  où  $r(X)$  est le reste de la division de  $a(X)$  par  $p(X)$ . Les coefficients de  $r(X)$  viennent à la place des  $k$  premiers 0 de  $a$ .

On aura bien ainsi  $Hb = (0)$  en effet  $Hb = Ha + H^t \begin{pmatrix} r_0 & r_1 & \dots & r_{k-1} & 0 & \dots & 0 \end{pmatrix}$  et  $Ha, H^t \begin{pmatrix} r_0 & r_1 & \dots & r_{k-1} & 0 & \dots & 0 \end{pmatrix} = \begin{pmatrix} r_0 & r_1 & \dots & r_{k-1} & 0 & \dots & 0 \end{pmatrix}$  sont égaux car  $a(X) \equiv r(X)[p(X)]$  et en base 2, on a  $1 + 1 = 0 + 0 = 0$ .

Le calcul de  $r(X)$  vient du calcul de  $Ha$ .

Une fois reçu le bloc  $c$ , le décodage consiste à calculer comme précédemment le vecteur  $s = Hc$ . Si  $s = 0$ , on garde  $c$ . Sinon, on modifie la composante  $i$  de  $c$  telle que  $s(x) \equiv x^i[p(X)]$ .

7. Écrire une fonction `hammingCycl(h, aPrime)` qui retourne le codage  $b$  de  $aPrime$ . Le paramètre  $h$  est un tableau numpy représentant la matrice de transition de taille  $(k, 2^k - 1)$  et  $aPrime$  est un tableau numpy représentant le bloc à coder de taille  $(1, 2^k - 1 - k)$ . Le résultat  $b$  est de taille  $(1, 2^k - 1)$ .  
Vérifier que  $[1, 1, 1, 0]$  est codé en  $[0, 1, 0, 1, 1, 1, 0]$ .
8. On suppose que le tableau numpy  $c$  est codé selon Hamming cyclique et qu'il contient au plus une erreur. On note  $h$  le tableau numpy de transition. Écrire une fonction `decodageHamCycl(h, c)` qui retourne  $c$  s'il n'y a pas d'erreur et une version corrigée sinon.

En doublant la taille de la matrice  $H$ , on peut donner une méthode qui peut détecter et corriger jusqu'à deux erreurs.

On voit dans cette version de Hamming l'importance du calcul arithmétique sur les polynômes. On va donc développer ces techniques dans les questions suivantes. Le codage de Reed-Solomon utilisé par les lecteurs DVD et les liaisons satellites exploite aussi la puissance des polynômes.

9. **Question ouverte** : À tout polynôme de  $\mathbb{R}[X]$  noté  $P(X) = \sum_{k=0}^n a_k X^k$ , on associe une liste  $[a_0, a_1, \dots, a_n]$  que l'on appelle *représentant* de  $P(X)$ . Le polynôme nul a pour représentant la liste vide  $[]$ .  $P(X)$  a plusieurs représentants, il suffit d'ajouter des 0 en fin de liste, par exemple,  $[a_0, a_1, \dots, a_n, 0, 0]$ .  
Écrire une fonction récursive `quoEtresteRec(a, b)` qui retourne un couple  $(q, p)$  où  $q$  représente le quotient de la division de  $A(X)$  par  $B(X)$  et  $p$  représente le quotient du reste de la division de  $A(X)$  par  $B(X)$ .

## 5 Polynôme

À tout polynôme de  $\mathbb{R}[X]$  noté  $P(X) = \sum_{k=0}^n a_k X^k$ , on associe une liste  $[a_0, a_1, \dots, a_n]$  que l'on appelle *représentant* de  $P(X)$ . Le polynôme nul a pour représentant la liste vide  $[]$ .  $P(X)$  a plusieurs représentants, il suffit d'ajouter des 0 en fin de liste, par exemple,  $[a_0, a_1, \dots, a_n, 0, 0]$ .

Dans le fichier `polynome.py` vous trouverez toutes les fonctions de cette partie sauf celles concernant les calculs de quotient et de reste. S'il vous reste peu de temps, vous pouvez l'utiliser.

### 5.1 Calibrage

10. \* Écrire une fonction `estnul(p)` qui retourne `True` si la liste  $p$  représente le polynôme nul et `False` sinon.
11. Écrire une fonction `deg(p)` qui retourne le degré du polynôme que  $p$  représente. Le polynôme nul aura pour degré -1.
12. Écrire une fonction `simplifie(p)` qui retourne  $[]$  si  $p$  représente le polynôme nul et sinon retourne ce qu'il reste de la liste  $p$  quand on a supprimé tous les 0 à la fin de  $p$ .

## 6 Évaluation

### 6.1 Approche naïve

13. \* Écrire une fonction `puissance(x,k)` qui retourne  $x^k$  avec  $x$  un réel et  $k \in \mathbb{N}$ . Seules les multiplications sont permises.
14. \* Écrire une fonction `eval_naif(x,p)` qui retourne  $P(x) = \sum_{k=0}^n a_k x^k$ , avec  $x$  un réel.

### 6.2 Algorithme de Horner

15. On pose  $u_0 = a_n$  et, pour tout  $k \in \llbracket 1, n \rrbracket$ ,  $u_k = a_{n-k} + x \times u_{k-1}$ .
  - (a) \* Calculer  $u_1, u_2$ . Que vaut  $u_n$  ?
  - (b) \* En déduire une fonction `horner(x,p)` qui retourne  $P(x) = \sum_{k=0}^n a_k x^k$ .
  - (c) \* Comparer sa complexité avec l'algorithme naïf.

## 7 Opération

16. Soit  $P(X)$  représenté par une liste  $p$  et  $Q(X)$  représenté par une liste  $q$ .
  - (a) Écrire une fonction `addition(p,q)` qui retourne une liste représentant  $P(X) + Q(X)$ .
  - (b) Soit  $a \in \mathbb{R}$ . Écrire une fonction `multi_scalaire(a,p)` qui retourne une liste représentant  $aP(X)$ .
  - (c) Écrire une fonction `soustraction(p,q)` qui retourne une liste représentant  $P(X) - Q(X)$ .
  - (d) Écrire une fonction `multi_monome(p,a,k)` qui retourne une liste représentant  $aX^k P(X)$ .
  - (e) Écrire une fonction `produit(p,q)` qui retourne une liste représentant  $P(X).Q(X)$ .
  - (f) \* Écrire une fonction `derivation(p)` qui retourne une liste représentant  $P'(X)$ .
  - (g) \* Écrire une fonction `integration(p,c)` qui retourne une liste représentant la primitive de  $P(X)$  qui vaut  $c$  en 0.

## 8 Arithmétique

Soit  $A(X)$  représenté par une liste  $a$  et  $B(X)$  représenté par une liste  $b$ .

17. Écrire une fonction itérative `quoEtreste(a,b)` qui retourne un couple  $(q, p)$  où  $q$  représente le quotient de la division de  $A(X)$  par  $B(X)$  et  $p$  représente le quotient du reste de la division de  $A(X)$  par  $B(X)$
18. Écrire une fonction récursive `quoEtresteRec(a,b)` qui retourne un couple  $(q, p)$  où  $q$  représente le quotient de la division de  $A(X)$  par  $B(X)$  et  $p$  représente le quotient du reste de la division de  $A(X)$  par  $B(X)$

Par exemple

```
>>> quo_et_reste([0., -9., 9., 0., 6., 4.], [-3., 3., 2.])
([0, 3.0, 0, 2.0], [])
>>> quoEtresteRec([3, 7, -3, 7, 9, 4], [4, 3, -1])
([-339.0, -86.0, -21.0, -4.0], [1359.0, 1368.0, 0.0])
>>> quo_et_reste([3, 7, -3, 7, 9, 4], [4, 3, -1])
([-339.0, -86.0, -21.0, -4.0], [1359.0, 1368.0])
```

## 9 Application

19. Écrire une fonction `matriceTransition(k,p)` qui retourne un tableau numpy  $h$  de taille  $(k, 2^k - 1)$  dont les colonnes sont les coordonnées des restes des divisions euclidiennes des polynômes  $(X^i)_{0 \leq i \leq 2^k - 2}$  par un polynôme primitif  $p(X)$  de degré  $k$  présenté sous forme de liste, les coefficients valent 0 ou 1. Les coordonnées sont données dans la base  $(X^j)_{0 \leq j \leq k-1}$ . Les coefficients sont dans  $\mathbb{Z}/2\mathbb{Z}$  avec les règles de calculs qui s'y rapportent.

```

import numpy as np
t=np.array([[8,5],[7,0]])
print(t)
y=np.zeros((2,4),int)
print(y)
print(y.shape)
l=[6,4,3,1,5,2]
y[0,1:3]=l[2:4]
print(y)
b=np.array([[2],[4],[6],[8]])
print(np.dot(y,b))
enfin=np.transpose(b)
print(enfin)

t1=np.array([[0,1,0,1],[1,1,1,0]])
t2=np.array([[0,1,0,1],[1,1,0,0]])
#Q1
def parite(liste):
    return(sum(liste)%2)
#Q2
def codage(liste,n,p):
    t=np.zeros((n,p),int)
    for k in range(n):
        t[k,0:p-1]=liste[k*(p-1):(k+1)*(p-1)]
        t[k,p-1]=parite(liste[k*(p-1):(k+1)*(p-1)])
    return(t)

#Q3
def verification(tableau):
    (n,p)=tableau.shape
    k=0
    while (k<n) and (parite((tableau[k,0:(p-1)]))==tableau[k,p-1]):
        k=k+1
    if k==n:
        return(-1)
    else :
        return(k)

#Q4
def codageHamming(b):
    return(np.array([(b[0]+b[1]+b[3])%2,(b[0]+b[2]+b[3])%2,b[0],(b[2]+b[1]+b[3])%2,b[1],b[2],b[3]]))

#Q5a
def matriceTrans(k):
    n=2**k-1
    p=1 # puissance de 2
    liste=[j for j in range(1,n+1)]
    h=np.zeros((k,n),int)
    for i in range(k) :
        for j in range(n):
            h[i,j]=(liste[j]//p)%2
        p=p*2
    return(h)

#Q5b

```

```

def codageHam2(k,b):
    p=2 #puissance de 2
    a=[0]+b
    for j in range(1,k): #on insere 0 en indice p-1
        a=a[0:p-1]+[0]+a[p-1:]
        p=2*p
    #On va placer les bonnes valeurs des ai
    p=1
    h=matriceTrans(k) # calcul de la matrice H
    col=np.dot(h,np.transpose(np.array(a))) # produit matriciel
    for j in range(k):
        a[p-1]=col[j]%2 # attention aux indices et aux calculs modulo 2
        p=2*p
    return(a)

#Q6
def decodageHam(h,a):
    col=np.dot(h,np.transpose(a))
    (k,n)=h.shape
    (i,)=col.shape
    for ligne in range(i):
        col[ligne]=col[ligne]%2
    nul=np.zeros((i,1),int)
    if list(col)==list(nul) :
        return(a)
    else :
        colonne=0
        while colonne< n and list(col) != list(h[:,colonne] ):
            colonne=colonne+1
        a[colonne]=(a[colonne]+1)%2
        return(a)

#Q7
def hammingCycl(h,aPrime):
    (k,n)=h.shape
    b=np.zeros((1,n),int)
    b[0,k:]=aPrime
    r=np.dot(h,np.transpose(b))
    r=list(r)
    b[0,0:k]=r
    for i in range(k):
        b[0,i]=b[0,i]%2
    return(b)

#Q8
def decodageHamCycl(h,c):
    col=np.dot(h,np.transpose(c))
    (k,n)=h.shape
    (i,)=col.shape
    for ligne in range(i):
        col[ligne]=col[ligne]%2
    nul=np.zeros((i,1),int)
    if list(col)==list(nul) :
        return(c)
    else :

```



```
        colonne=0
        while colonne< n and list(col) != list(h[:,colonne] ):
            colonne=colonne+1
        c[colonne]=(c[colonne]+1)%2
        return(c)

#Q

#Q

#Q

def estnul(p) :
    if p==[] :
        return(True)
    else :
        n=len(p)
        test=True
        k=0
        while test and (k<n) :
            if p[k]==0 :
                k=k+1
            else :
                test=False
        return(test)

##### parce que Python : #####

def isnul(p) :
    return(sum([abs(x) for x in p]) ==0)

def deg(p) :
    if estnul(p) :
        return(-1)
    else :
        n=len(p)
        k=n-1
        while p[k]==0 :
            k=k-1
        return(k)

def val(p) :
    if estnul(p) :
        return(-2)
    else :
        k=0
        while p[k]==0 :
            k=k+1
        return(k)

def simplifie(p) :
    d=deg(p)
    if d==-1 :
```

```

        return([])
    else :
        return(p[0:d+1])

#####II Evaluation#####
def puissance(a,k) :
    puissance=1
    for j in range(k) :
        puissance=puissance*a
    return(puissance)

def eval_naif(a,p) :
    if estnul(p) :
        return(0)
    else :
        e=p[0]
        for k in range(1,deg(p)+1):
            e=e + p[k]*puissance(a,k)
        return(e)

def horner(a,p) :
    if estnul(p) :
        return(0)
    else :
        d=deg(p)
        e=p[d]
        for k in range(d):
            e=p[d-1-k]+ a*e
        return(e)

##### III Op\'erations #####

def addition(p,q) :
    dp=deg(p)
    dq=deg(q)
    if dp==-1 :
        return(q)
    elif dq==-1 :
        return(p)
    elif dp==dq :
        l=[]
        for k in range(dp+1) :
            l.append(p[k]+q[k])
        return(l)
    elif dp<dq :
        l=[]
        for k in range(dp+1) :
            l.append(p[k]+q[k])
        l=l+q[dp+1:dq+1]
        return(l)
    else :
        l=[]
        for k in range(dq+1) :
```

```
        l.append(p[k]+q[k])
    l=l+p[dq+1:dp+1]
    return(l)

def multi_scalaire(a,p) :
    if (a==0) or estnul(p) :
        return([])
    else :
        return([a*x for x in p])

def soustraction(p,q) :
    return(addition(p,multi_scalaire(-1,q)))

def multi_monome(p,a,k) :
    if isnul(p) :
        return([])
    else :
        return([0]*k+multi_scalaire(a,p))

def produit(p,q) :
    if estnul(p) or estnul(q) :
        return([])
    else :
        dp=deg(p)
        dq=deg(q)
        d=dp+dq
        p1=p+[0]*(d-dp)
        p2=q+[0]*(d-dq)
        l=[]
        for i in range(d+1) :
            c=0
            for j in range(i+1) :
                c=c+p1[j]*p2[i-j]
            l.append(c)
        return(l)

def derivation(p) :
    if deg(p)<1 :
        return([])
    else :
        l=[]
        for k in range(1,deg(p)+1) :
            l.append(k*p[k])
        return(l)

def integration(p,c) :
    d=deg(p)
    if d==-1 :
        return(c)
    else :
        l=[c]
        for k in range(0, d+1) :
            l.append(p[k]/(k+1))
        return(l)
```

```
##### IV arithm\’etique #####
```

```
def quo_et_reste(a,b) :
    dd=deg(a)
    db=deg(b)
    if db>dd :
        return([],a)
    else :
        q=[]
        dividende=list(a)
        while (dd>= db) :
            q=addition(q, [0]*(dd-db)+[dividende[dd]/b[db]])
            dividende=soustraction(dividende,multi_monome(b,dividende[dd]/b[db],dd-db))
            dd=deg(dividende)
        return(q,simplifie(dividende))

quo_et_reste([3,7,-3,7,9,4],[4,3,-1])

quo_et_reste([0,-9,9,0,6,4],[-3,3,2])

def quoEtResteRec(a,b):
    da=deg(a)
    db=deg(b)
    if da<db :
        return([],a)
    else :
        c=a[da]/b[db]
        (qTemp,r)=quoEtResteRec(soustraction(a,multi_monome(b,c,da-db)),b)
        return(addition(qTemp,[0]*(da-db)+[c]),r)

#Q18
def resteZ2Z(a,b):
    '''calcul du reste modulo 2'''
    (q,r)=quoEtResteRec(a,b)
    d=len(r)
    for i in range(d):
        r[i]=int(r[i])%2
    return(r)

def matriceTransition(k,p) :
    n=2**k-1
    h=np.zeros((k,n),int)
    for j in range(n) :
        r=resteZ2Z([0]*j+[1],p)
        r=r+[0]*(k-len(r)) # donne une bonne longueur a r
        for i in range(k) :
            h[i,j]=r[i] # on remplit la colonne j
    return(h)
```